

Divide and Conquer

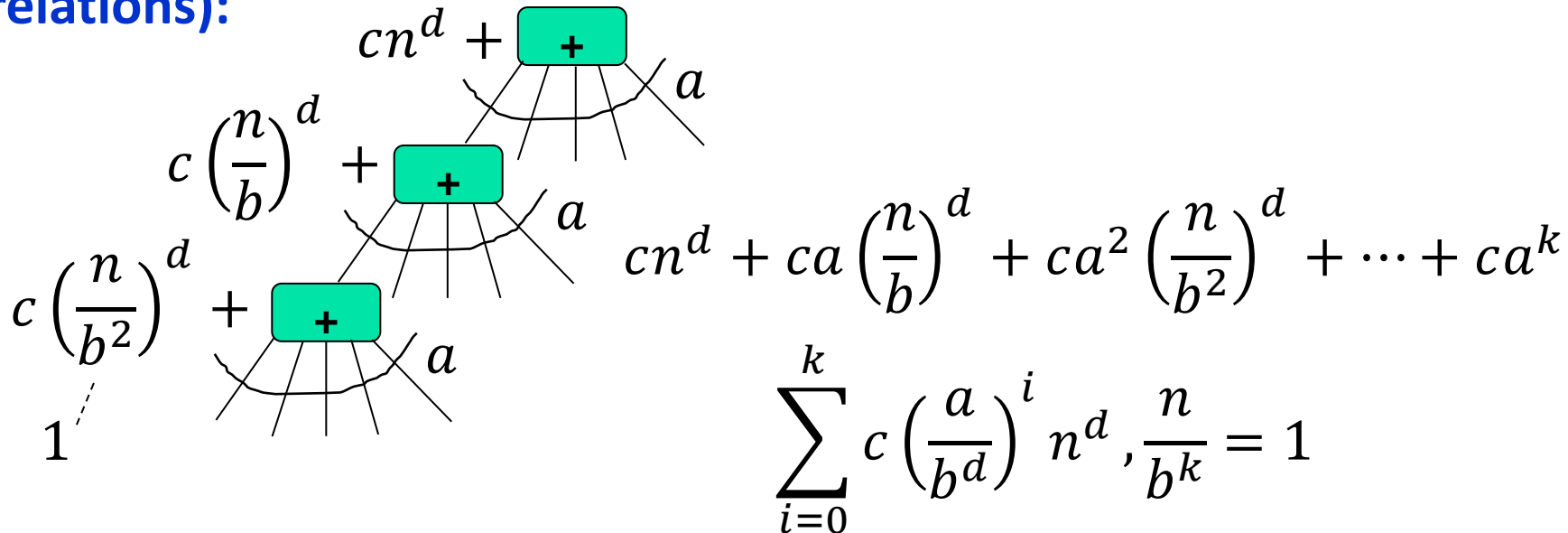
Karatsuba's Algorithm! $\longrightarrow O(n^{\log_2 3})$

$$T(n) = 3T\left(\frac{n}{2}\right) + cn$$

In general:

$$T(n) = aT\left(\frac{n}{b}\right) + cn^d$$

**Recursion tree
(relations):**



$$1 + p + p^2 + \dots + p^k = \sum_{i=0}^k (p)^i = \begin{cases} k + 1, & \text{if } p = 1 \\ \frac{p^{k+1} - 1}{p - 1}, & \text{if } p \neq 1 \end{cases}$$

geometric series

$$\sum_{i=0}^k c \left(\frac{a}{b^d}\right)^i n^d = cn^d \sum_{i=0}^{\log_b(n)} \left(\frac{a}{b^d}\right)^i = \begin{cases} cn^d (\log_b(n) + 1), & \text{if } \frac{a}{b^d} = 1 \\ cn^d \frac{\left(\frac{a}{b^d}\right)^{\log_b(n)+1} - 1}{\left(\frac{a}{b^d}\right) - 1}, & \text{if } \frac{a}{b^d} \neq 1 \end{cases}$$



$$cn^d \frac{\left(\frac{a^{\log_b(n)+1}}{(b^d)^{\log_b(n)+1}}\right) - 1}{\left(\frac{a}{b^d}\right) - 1} = \frac{cn^d \left[\frac{a}{b^d} \left(\frac{(b^{\log_b(a)})^{\log_b(n)}}{(b^{\log_b(n)})^d} \right) - 1 \right]}{\left(\frac{a}{b^d}\right) - 1}$$

$$= c \frac{\frac{a}{b^d} ((b^{\log_b(n)})^{\log_b(a)}) - n^d}{\left(\frac{a}{b^d}\right) - 1} = c \frac{a(n^{\log_b(a)}) - n^d b^d}{a - b^d}$$

$$a > b^d, a < b^d$$

$$a < b^d \rightarrow \log_b a < d$$

$$c \frac{a(n^{\log_b(a)}) - n^d b^d}{a - b^d} = x(n^d b^d - a(n^{\log_b(a)})), x = \frac{c}{b^d - a}$$

$$a > b^d \rightarrow \log_b a > d$$

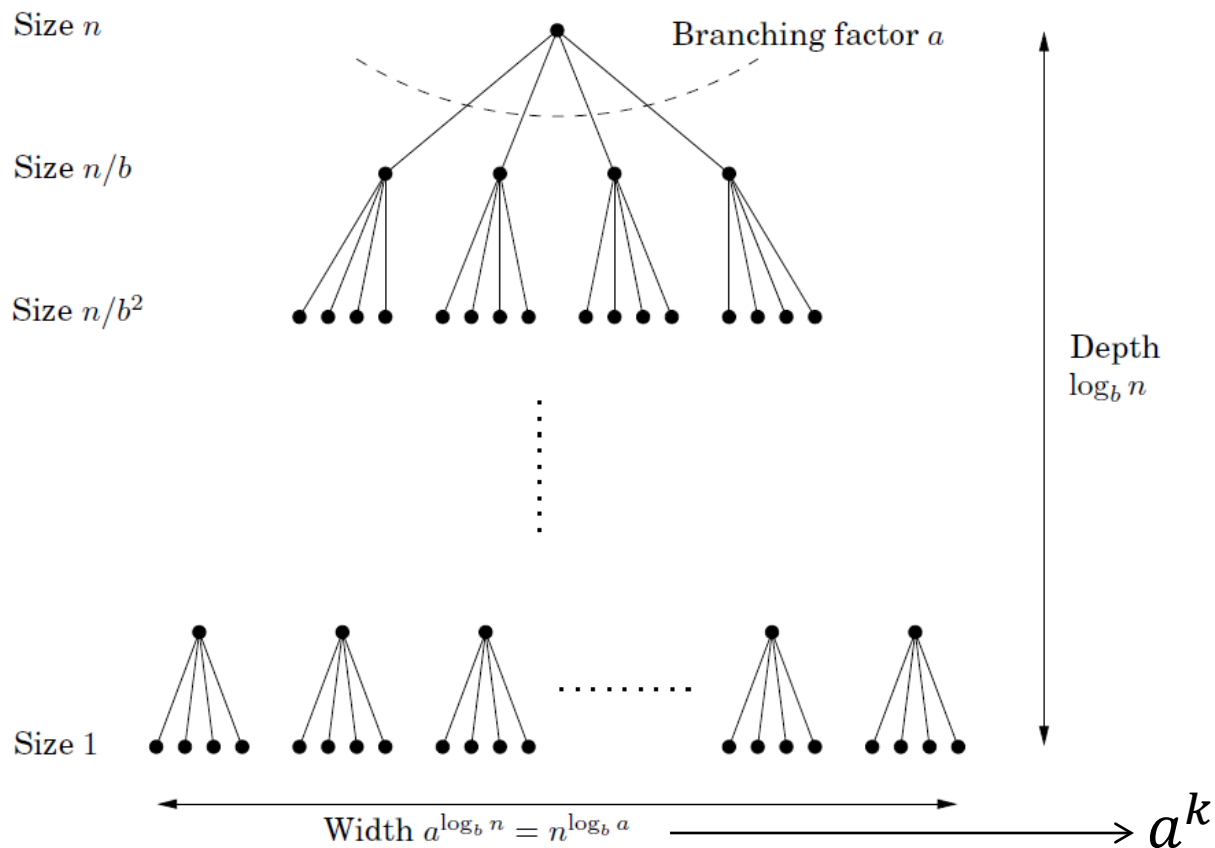
$$c \frac{a(n^{\log_b(a)}) - n^d b^d}{a - b^d} = x(a(n^{\log_b(a)}) - n^d b^d), x = \frac{c}{a - b^d}$$

$$\text{Time complexity / Efficiency} \quad \left\{ \begin{array}{ll} O(n^d \log_b(n)), \text{ if } \frac{a}{b^d} = 1 & \text{Case E} \\ O(n^d), \text{ if } \frac{a}{b^d} < 1 & \text{Case L} \\ O(n^{\log_b(a)}), \text{ if } \frac{a}{b^d} > 1 & \text{Case G} \end{array} \right.$$

MASTER THEOREM

More general recursive relation is not represented.

Each problem of size n is divided into a subproblems of size n/b .



Master theorem If $T(n) = aT(\lceil n/b \rceil) + O(n^d)$ for some constants $a > 0$, $b > 1$, and $d \geq 0$, then

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a . \end{cases}$$

- $T(n) = T(n - 1) + c^n.$

$$= T(n - 2) + c^{n-1} + c^n$$

$$\vdots$$

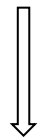
$$= T(1) + \dots + c^{n-1} + c^n \Rightarrow O(c^n)$$

- $T(n) = T(\sqrt{n}) + 1.$

$$= T(n^{1/4}) + 2$$

$$\vdots$$

$$= T(n^{1/2^k}) + k$$



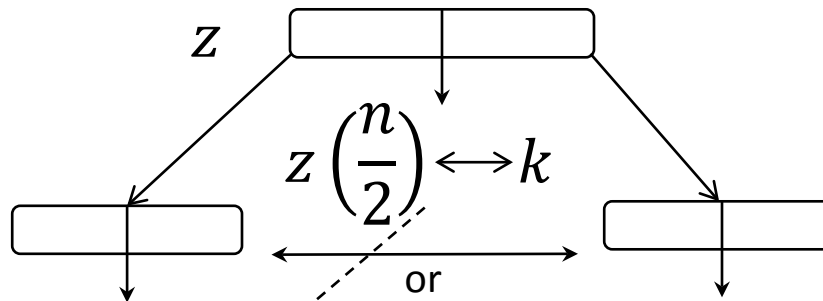
Will be constant when $k = \log_2(\log_2(n))$

$$\xrightarrow{n^{1/2^k} = 2}$$

$$= C + \log_2(\log_2(n)) \Rightarrow O(\log(\log(n)))$$

Example, Binary Search

Find a key k in the sorted $z[0, 1, \dots, n - 1]$:



$$T(n) = T\left(\frac{n}{2}\right) + 1$$

Master theorem with $a = 1, b = 2, c = 1, d = 0 \Rightarrow O(\log n)$

Matrix Multiplication

product of two $n \times n$ matrices X and Y

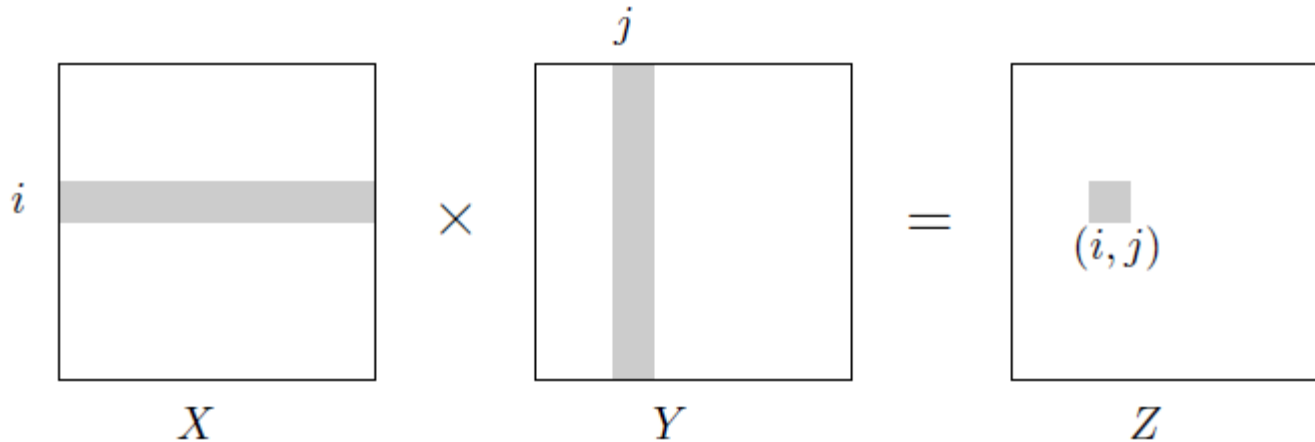
$$Z = XY$$

$\downarrow$
 $n \times n$ matrix

$$Z_{ij} = \sum_{k=1}^n X_{ik} Y_{kj}$$

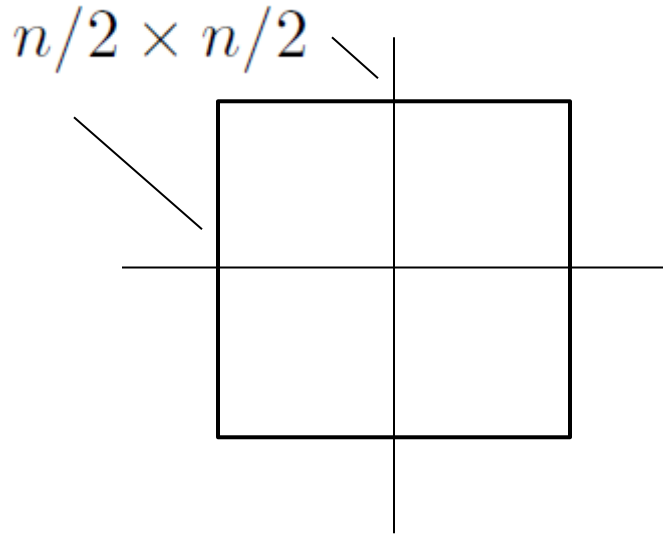
$n^2 \times n \longrightarrow O(n^3)$

If not square?



Again, it was believed for long that this was the best you can do!

Strassen – 1969!



$$X = \begin{bmatrix} A & B \\ C & D \end{bmatrix}, \quad Y = \begin{bmatrix} E & F \\ G & H \end{bmatrix}$$

$$\begin{aligned} XY &= \begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} E & F \\ G & H \end{bmatrix} \\ &= \begin{bmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{bmatrix} \end{aligned}$$

$$T(n) = 8T(n/2) + O(n^2)$$

Master theorem with $a = 8, b = 2, d = 2$

$$T(n) = O(n^{\log_2 8}) = O(n^3)$$

But:

$$XY = \begin{bmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_1 + P_5 - P_3 - P_7 \end{bmatrix}$$

$$P_1 = A(F - H) \quad P_5 = (A + D)(E + H)$$

$$P_2 = (A + B)H \quad P_6 = (B - D)(G + H)$$

$$P_3 = (C + D)E \quad P_7 = (A - C)(E + F)$$

$$P_4 = D(G - E)$$

$$T(n) = 7T(n/2) + O(n^2) = O(n^{\log_2 7}) = O(n^{2.81})$$

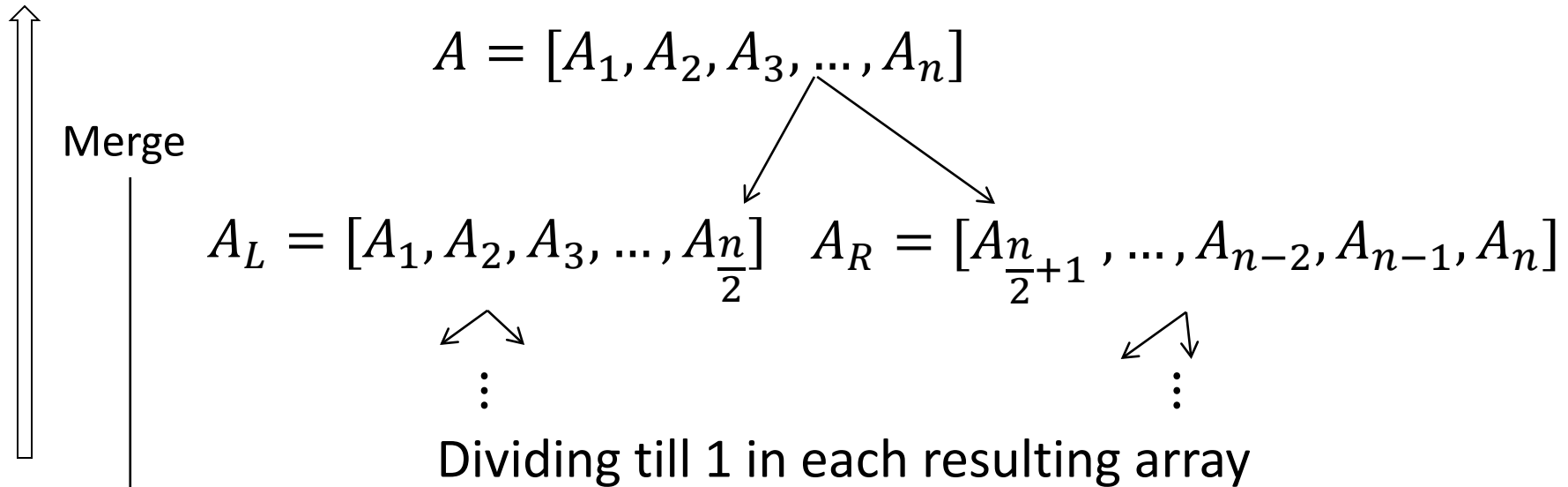
Further evolution:

$$1987 - O(n^{2.375477})$$

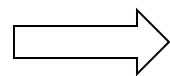
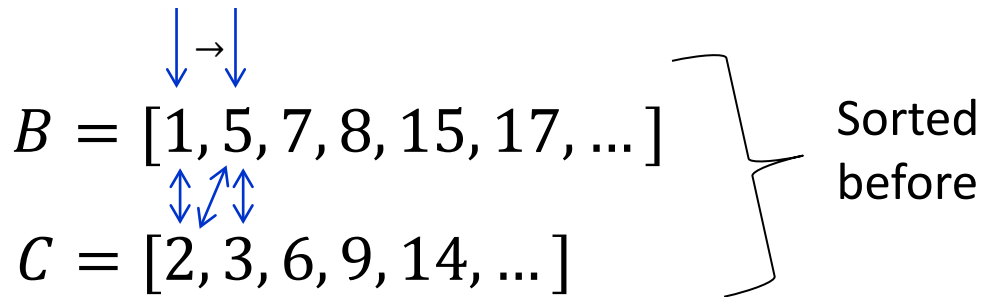
$$2010 - O(n^{2.37368}) \quad 2012 - O(n^{2.372873}) \quad 2014 - O(n^{2.372864})$$

1987 paper: <https://dl.acm.org/doi/10.1145/28395.28396>

Sorting (Mergesort)



Example:



$[1, 2, 3, 5, \dots]$

$$\approx 2 \left(\frac{n}{2^i} \right)$$

These many comparisons between the numbers at the pointers

Linear in n at that level ($n(i) \sim n/2^{i-1}$)

The algorithm:

function mergesort ($a[1 \dots n]$)

Input: An array of numbers $a[1 \dots n]$

Output: A sorted version of this array

if $n > 1$:

 return merge(mergesort($a[1 \dots \lfloor n/2 \rfloor]$), mergesort($a[\lfloor n/2 \rfloor + 1 \dots n]$))

else:

 return a

function merge ($x[1 \dots k], y[1 \dots l]$)

if $x[1] \leq y[1]$: $\longrightarrow$ If $k == 1$: return $x[1] \circ y[1 \dots l]$ else:
 return $x[1] \circ \text{merge}(x[2 \dots k], y[1 \dots l])$ $\circ$ denotes concatenation

else: $\longrightarrow$ If $k == 1$: return $y[1] \circ x[1 \dots k]$ else:
 return $y[1] \circ \text{merge}(x[1 \dots k], y[2 \dots l])$

$$T(n) = 2T(n/2) + O(n)$$

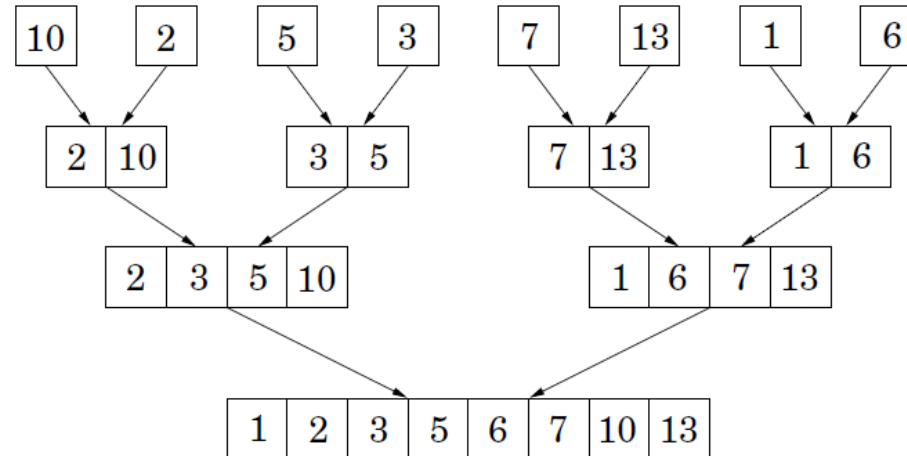
$$a = 2, b = 2, d = 1$$

$$O(n \log n)$$

Input:

10	2	5	3	7	13	1	6
----	---	---	---	---	----	---	---

An example:



Iteration instead of recursion... will be faster?

```
function iterative-mergesort(a[1...n])  
Input:  elements  $a_1, a_2, \dots, a_n$  to be sorted
```

```
 $Q = [ ]$  (empty queue)
```

```
for  $i = 1$  to  $n$ :
```

```
    inject( $Q, [a_i]$ )
```

```
while  $|Q| > 1$ :
```

```
    inject( $Q, \text{merge}(\text{eject}(Q), \text{eject}(Q))$ )
```

```
return eject( $Q$ )
```

... no

$O(n \log n)$

Space complexity of Mergesort? $O(n)$

Timsort... best case time $O(n)$ average/worst $O(n \log n)$, space $O(n)$

Heapsort... time $O(n \log n)$, space $O(\log n)$ or $O(1)$

Fastest sorting possible?

In 2002 Deterministic - $O(n \log(\log(n)))$

<https://dl.acm.org/doi/10.1145/509907.509993>

Randomized (expected) - $O\left(n\sqrt{\log(\log(n))}\right)$

<https://dl.acm.org/doi/10.5555/645413.652131>

$O(n \log(n))$ Is the best you can achieve
by doing only comparisons!

Proof next...

$O(n \log(n))$ Is the best you can achieve
by doing only comparisons!

Example of sorting using only comparisons: a_1, a_2, a_3

Binary
decision
tree

